

当前版本的反馈，基于反馈调整产品，例如，进行调试、性能或可用性的增强等。

移交阶段结束时也要进行技术评审，评审目标是否实现、是否应该开始演化过程、用户对交付的产品是否满意等。

从本节的介绍可以看出，RUP 由于太过庞大和复杂，相对于轻量级的敏捷方法来说，显得死板和难以实施。RUP 不但不能快速适应需求的变化，而且变更一个需求要经历复杂的过程和很多额外的工作。对于较小的组织和项目来说，使用敏捷方法可能比较合适，而使用 RUP 似乎有些费力不讨好。

7.2.4 敏捷方法

敏捷方法是从 20 世纪 90 年代开始引起广泛关注的一些新型软件开发方法，以应对快速变化的需求。虽然它们的具体名称、理念、过程、术语都不尽相同，但相对于“非敏捷”而言，它们更强调开发团队与用户之间的紧密协作、面对面的沟通、频繁交付新的软件版本、紧凑而自我组织型的团队等，也更注重人的作用。

1. 敏捷宣言

2001 年，Kent Beck 等人组织了敏捷联盟，阐述了敏捷开发的原则，试图强调灵活性在快速且有效地开发软件中所发挥的作用，他们共同签署了敏捷软件开发宣言，该宣言认为，个体和交互胜过过程和工具；可工作的软件胜过大量的文档；客户合作胜过合同谈判；响应变化胜过遵循计划。

敏捷方法强调：让客户满意和软件尽早增量发布；小而高度自主的项目团队；非正式的方法；最小化软件工作产品以及整体精简开发。产生这种情况的原因是，在绝大多数软件开发过程中，提前预测哪些需求是稳定的和哪些需求会变化非常困难；对于软件项目构建来说，设计和实现是交错的；从制定计划的角度来看，分析、设计、实现和测试并不容易预测；可执行原型和部分实现的可运行系统是了解用户需求和反馈的有效媒介。

目前，主要的敏捷方法有极限编程（eXtreme Programming, XP）、自适应软件开发（Adaptive Software Development, ASD）、水晶方法（Crystal）、特性驱动开发（Feature-Driven Development, FDD）、动态系统开发方法（Dynamic Systems Development Method, DSDM）、测试驱动开发（Test-Driven Development, TDD）、Scrum、敏捷数据库技术（Agile Database Techniques, AD）和精益软件开发（Lean Software Development）等。虽然这些过程模型在实践上有差异，但都遵循敏捷宣言或者敏捷联盟所定义的基本原则。这些原则包括客户参与、增量式移交、简单性、接受变更、强调开发人员的作用和及时反馈等。

2. 敏捷方法的特点

敏捷方法是一种以人为核心、迭代、循序渐进的开发方法。在敏捷方法中，软件项目的构建被切分成多个子项目，各个子项目的成果都经过测试，具备集成和可运行的特征。在敏捷方法中，从开发者的角度来看，主要的关注点有短平快的会议、小版本发布、较少的文档、合作为重、客户直接参与、自动化测试、适应性计划调整和结对编程；从管理者的角度来看，主要的关注点有测试驱动开发、持续集成和重构。

近年来，虽然敏捷方法发展得较快，但在实施的过程中，也暴露出很多问题，一些敏捷方法的基本原则很难实施，主要体现在以下4个方面：

- (1) 客户参与往往依赖于客户参与的意愿和客户自身的代表性。
- (2) 团队成员的性格可能不适合激烈的投入，可能无法做到与其他成员之间良好沟通。
- (3) 对系统的变更做出优先级排序可能是极端困难的。

(4) 维护系统的简洁性往往需要额外的工作，但迫于时间表的压力，可能没有时间执行系统简化过程。

与 RUP 相比，敏捷方法的周期可能更短。敏捷方法在几周或几个月的时间内完成相对较小的功能，强调的是能尽早将尽量小的可用的功能交付使用，并在整个项目周期中持续改善和增强，并且更加强调团队中的高度协作。相对而言，敏捷方法主要适用于以下场合：

(1) 项目团队的人数不能太多，适合于规模较小的项目。

(2) 项目经常发生变更。敏捷方法适用于需求萌动并且快速改变的情况，如果系统有比较高的关键性、可靠性、安全性方面的要求，则可能不完全适合。

(3) 高风险项目的实施。

(4) 从组织的角度看，组织的文化、人员、沟通性决定了敏捷方法是否适用。与这些相关联的关键成功因素有组织文化必须支持谈判、人员彼此信任、人少但是精干、开发人员所做的决定得到认可、环境设施满足团队成员之间快速沟通的需要。

3. 极限编程方法

敏捷方法中最著名的就是极限编程（XP），XP 是一种轻量、高效、低风险、柔性、可预测、科学且充满乐趣的软件开发方式，适用于小型或中型软件开发团队，并且客户的需求模糊或需求多变。与其他方法相比，XP 方法最大的不同如下：

(1) 在更短的周期内，更早地提供具体、持续的反馈信息。

(2) 迭代地进行计划编制，首先在最开始迅速生成一个总体计划，然后在整个项目开发过程中不断地发展它。

(3) 依赖于自动测试程序来监控开发进度，并尽早捕获缺陷。

(4) 依赖于口头交流、测试和源程序进行沟通。

(5) 倡导持续的演化式的设计。

(6) 依赖于开发团队内部的紧密协作。

(7) 尽可能达到程序员短期利益和项目长期利益的平衡。

XP 由价值观、原则、实践和行为 4 个部分组成，它们彼此相互依赖、关联，并通过行为贯穿于整个生命周期。XP 的核心是其总结的四大价值观，即沟通、简单、反馈和勇气。它们是 XP 的基础，也是 XP 的灵魂。XP 的 5 个原则是快速反馈、简单性假设、逐步修改、提倡更改和优质工作。而在 XP 方法中，贯彻的是“小步快走”的开发原则，因此工作质量决不可打折扣，通常采用测试先行的编码方式来提供支持。

4. 动态系统开发方法

动态系统开发方法（DSDM）倡导以业务为核心，快速而有效地进行系统开发。可以把

DSDM 看成一种控制框架，其重点在于快速交付并补充如何应用这些控制的指导原则。

DSDM 是一整套的方法论，不仅仅包括软件开发内容和实践，也包括了组织结构、项目管理、估算、工具环境、测试、配置管理、风险管理、重用等各个方面的内容。

1) DSDM 的基本观点

DSDM 认为任何事情都不可能一次性圆满完成，应该用 20% 的时间完成 80% 的有用功能，以适合商业目的为准。实施的思路是：在时间进度和可用资源预先固定的情况下，力争最大化地满足业务需求（传统方法一般是需求固定，时间和资源可变），交付所需要的系统。对于交付的系统，必须达到足够的稳定程度，以在实际环境中运行；对于业务方面的某些紧急需求，也必须能够在短时间内得到满足，并在后续迭代阶段对功能进行完善。

2) DSDM 的基本原则

DSDM 的基本原则包括：

- 用户必须参与。
- 必须授权 DSDM 团队进行决策。
- 注重频繁交付产品。
- 判断产品是否可接受的基本标准是是否符合业务目的。
- 对准确的业务解决方案需要采用循环和增量开发。
- 开发期间的所有更改都是可逆的。
- 在高层次上制定需求的基线（以在低优先级的项目上获得一定的灵活性）。
- 在整个生命周期集成测试。
- 在所有参与者之间采用协作和合作方法。

5. Scrum

Scrum 是一个用于开发和维护复杂产品的框架，是一个增量的、迭代的开发过程。在这个框架中，整个开发过程由若干个短的迭代周期组成，一个短的迭代周期称为一个 Sprint，每个 Sprint 的建议长度是 2 ~ 4 周。

在 Scrum 中，使用产品 Backlog 来管理产品的需求，产品 Backlog 是一个按照商业价值排序的需求列表，列表条目的体现形式通常为用户故事。Scrum 团队总是先开发对客户具有较高价值的需求。在 Sprint 中，Scrum 团队从产品 Backlog 中挑选最高优先级的需求进行开发。

挑选的需求在 Sprint 计划会议上经过讨论、分析和估算得到相应的任务列表，称为 Sprint Backlog。在每个迭代结束时，Scrum 团队将递交潜在可交付的产品增量。Scrum 起源于软件开发项目，但它适用于任何复杂的或创新性的项目。

7.3 软件开发环境与工具

软件开发环境（Software Development Environment，SDE）是指支持软件的工程化开发和维护而使用的一组软件，由软件工具集和环境集成机制构成。软件工具是指 CASE 工具，用以